Bayesian predictive projection for variable selection

Setup

Data

I set the true outcome prevalence to 0.3, the true c-statistic to 0.8 and the number of predictor variables to 30. Based on these settings, the recommended development sample size for an expected calibration slope of 0.9 was 1505. I set the development dataset to a size of 752, which was half of the recommended sample size, and set the validation dataset to a size of 10,000.

The development and validation datasets each contained a binary outcome and 30 continuous predictor variables. The predictor variables were generated from a multivariate normal distribution N(0, I_{30}). The predictor variables were categorized into four types according to their predictive effects: strong, medium, weak and noise, with corresponding relative effect sizes of 1, 0.5, 0.25 and 0 respectively. We specified that the relative proportions of strong, medium, weak and noise variables were 0.1, 0.2, 0.2, 0.5.

```
# development dataset of size 752
data.dev <- read.csv("data_dev.csv")
sam <- sample(1:752, 376)
data.dev <- data.dev[sam,]
x <- data.dev[,-1]
y <- data.dev[,1]

# validation dataset of size 10000
data.val <- read.csv("data_val.csv")
xval <- data.val[,-1]
yval <- data.val[,1]
```

Packages

I used the following packages through the analysis.

```
library(tidyverse)
library(rstanarm)
library(brms)
library(projpred)
library(tidybayes)
library(broom.mixed)
library(pROC)
library(patchwork)
library(cutpointr)
library(gt)
library(coda)
library(loo)
library(glmnet)
library(detectseparation)
library(brglm2)
library(foreach)
library(MASS)
options(mc.cores = parallel::detectCores())
```

Fit a reference model

I fitted a Bayesian logistic regression with all predictor variables using the stan_glm function. I set priors for the individual coefficients and the intercept as weakly informative normal distributions N(0,1).

```
SEED = 7
set.seed(SEED)
refform <- as.formula(paste("y", paste(colnames(data.dev)[-1], collapse=" + "), sep=" ~ "))
ref_fit <- stan_glm(formula = refform,
                    data = data.dev,
                    family = binomial(link = "logit"),
                    prior = laplace(location = 0, scale = 1),
                    prior_intercept = normal(0, 1),
                    QR = TRUE,
                    seed = SEED,
                    adapt_delta = 0.99,
                    iter = 4000,
                    cores = 4,
                    chains = 4)
```

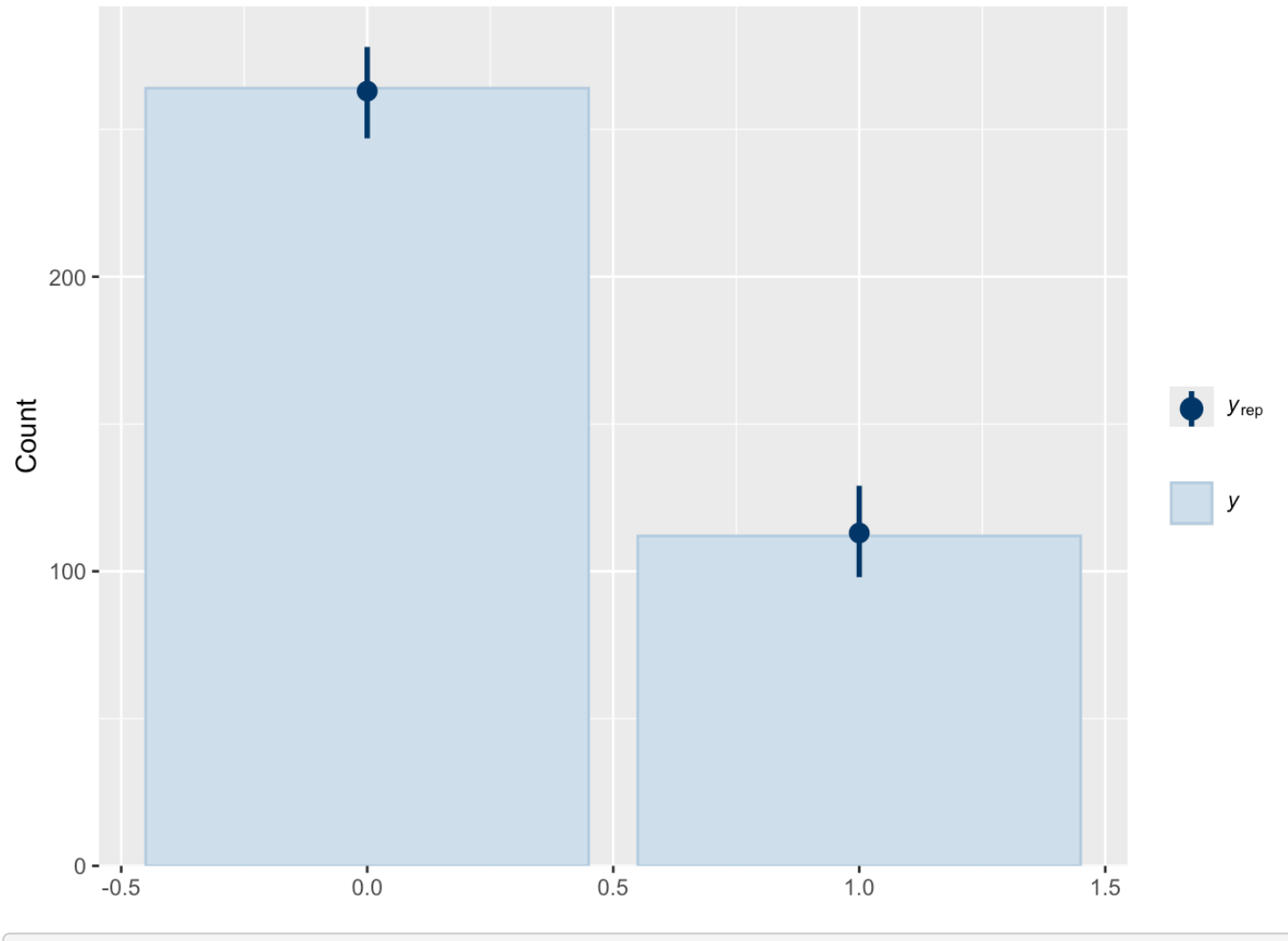
Check model diagnostics

```
summary(ref_fit)
```

```
## 
## Model Info:
##  function:     stan_glm
##  family:       binomial [logit]
##  formula:      y ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 + X11 + 
##     X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20 + X21 + 
##     X22 + X23 + X24 + X25 + X26 + X27 + X28 + X29 + X30
##  algorithm:    sampling
##  sample:       8000 (posterior sample size)
##  priors:       see help('prior_summary')
##  observations: 376
##  predictors:   31
## 
## Estimates:
##               mean   sd   10%   50%   90%
## (Intercept) -1.1    0.1 -1.3  -1.1  -0.9 
## X1           0.6    0.2  0.4   0.6   0.8 
## X2           0.5    0.2  0.3   0.5   0.7 
## X3           0.6    0.2  0.4   0.6   0.8 
## X4           0.2    0.1  0.0   0.2   0.3 
## X5           0.5    0.1  0.3   0.5   0.7 
## X6           0.2    0.1  0.0   0.2   0.4 
## X7           0.4    0.2  0.2   0.4   0.6 
## X8           0.5    0.1  0.3   0.5   0.6 
## X9           0.2    0.1  0.0   0.2   0.4 
## X10          0.1    0.1 -0.1   0.1   0.3 
## X11          0.2    0.1  0.0   0.2   0.4 
## X12          0.3    0.2  0.1   0.3   0.5 
## X13          0.2    0.1  0.0   0.1   0.3 
## X14          0.1    0.1  0.0   0.1   0.3 
## X15          0.0    0.1 -0.2   0.0   0.1 
## X16         -0.1    0.1 -0.2  -0.1   0.1 
## X17         -0.2    0.1 -0.4  -0.2   0.0 
## X18         -0.1    0.1 -0.3  -0.1   0.1 
## X19          0.0    0.1 -0.2   0.0   0.1 
## X20          0.1    0.1 -0.1   0.1   0.3 
## X21          0.3    0.1  0.1   0.3   0.5 
## X22          0.0    0.1 -0.2   0.0   0.2 
## X23          0.2    0.1  0.0   0.2   0.4 
## X24         -0.1    0.1 -0.3  -0.1   0.1 
## X25          0.2    0.1  0.0   0.2   0.3 
## X26          0.1    0.1 -0.1   0.1   0.3 
## X27          0.1    0.1 -0.1   0.1   0.2 
## X28          0.3    0.1  0.1   0.3   0.4 
## X29          0.0    0.1 -0.2   0.0   0.2 
## X30          0.2    0.1  0.0   0.2   0.3 
## 
## Fit Diagnostics:
##            mean   sd   10%   50%   90%
## mean_PPD 0.3    0.0  0.3   0.3   0.3  
## 
## The mean_ppd is the sample average posterior predictive distribution of the outcome variable (for details see help('summary.stanreg')).
## 
## MCMC diagnostics
##               mcse Rhat n_eff
## (Intercept)   0.0  1.0  10857
## X1            0.0  1.0   8413
## X2            0.0  1.0   8126
## X3            0.0  1.0   7676
## X4            0.0  1.0   7078
## X5            0.0  1.0   8258
## X6            0.0  1.0   7421
## X7            0.0  1.0   7703
## X8            0.0  1.0   8547
## X9            0.0  1.0   7364
## X10           0.0  1.0   8141
## X11           0.0  1.0   8332
## X12           0.0  1.0   8068
## X13           0.0  1.0   7848
## X14           0.0  1.0   8793
## X15           0.0  1.0   7519
## X16           0.0  1.0   7552
## X17           0.0  1.0   9152
## X18           0.0  1.0   7994
## X19           0.0  1.0   8176
## X20           0.0  1.0   8634
## X21           0.0  1.0   9038
## X22           0.0  1.0   8053
## X23           0.0  1.0   7695
## X24           0.0  1.0   7827
## X25           0.0  1.0   7777
## X26           0.0  1.0   7261
## X27           0.0  1.0   8288
## X28           0.0  1.0   7802
## X29           0.0  1.0   7152
## X30           0.0  1.0   8079
## mean_PPD      0.0  1.0   9164
## log-posterior 0.2  1.0   1817
## 
## For each parameter, mcse is Monte Carlo standard error, n_eff is a crude measure of effective sample size, and Rhat is the potential scale reduction factor on split chains (at convergence Rhat=1).
```

The model converged well, with no divergent transitions, good rhat values (all ~ 1), and large effective sample size (> 5,000) for all parameters. Next, I proceeded with a visual posterior predictive check and used PSIS-LOO cross validation to check for influential outliers.

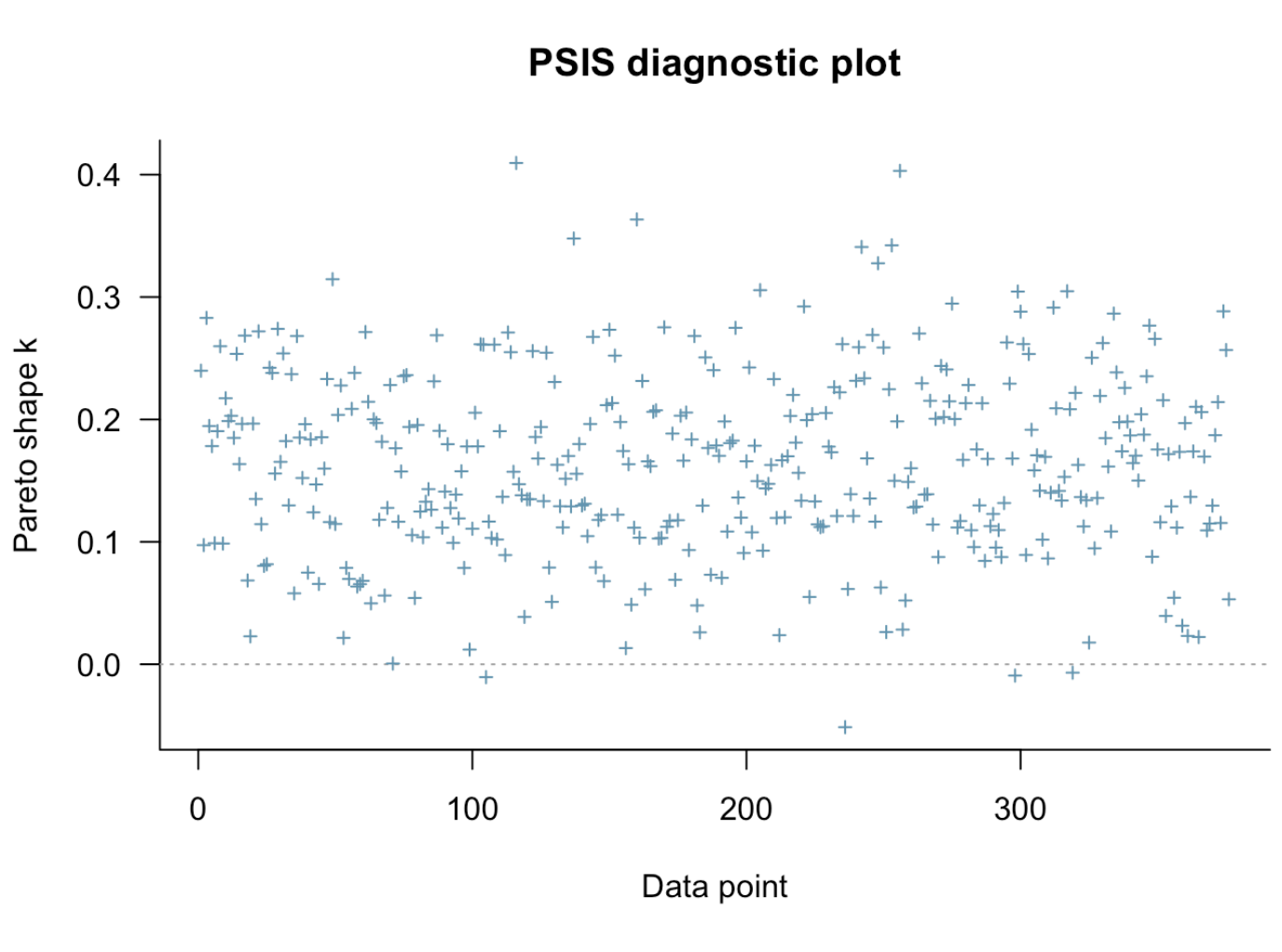
```
pp_check(ref_fit, plotfun = "bars", nreps = 500)
```



```
loo_reference_fit <- loo(ref_fit)
loo_reference_fit
```

```
## 
## Computed from 8000 by 376 log-likelihood matrix.
## 
##          Estimate   SE
## elpd_loo   -210.8 11.9
## p_loo        33.9  2.4
## looic       421.5 23.9
## ------
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
## 
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

```
plot(loo_reference_fit)
```



Using the posterior predictive check, the observed (training) data seemed to match the simulated draws (n = 500) from the posterior predictive distribution fairly well. All points had a Pareto shape parameter k < 0.5, suggesting that these observations did not exert an overly strong influence on the model fit.

Projection predictive variable selection

I performed an initial search through the model space, without cross-validation and using all available data to achieve a series of nested, increasingly complex submodels deemed the solution path. To do this, I used the cv_varsel function from the projpred package. This procedure identifies the submodel that yields predictions most similar to those of the reference model for each submodel size (defined as the number of predictors included).

```
  varsel_pro <- cv_varsel(ref_fit,
                    method = 'forward',
                    validate_search = F,
                    cores = 8,
                    seed = 11)
```

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnostic') for details.
```

After projection predictive variable selection, I checked the variable importance, as well the the suggested model size.

```
(vif <- ranking(varsel_pro)$fulldata)
```

```
##  [1] "X3"  "X1"  "X2"  "X8"  "X5"  "X7"  "X12" "X21" "X23" "X9"  "X28" "X6" 
## [13] "X17" "X11" "X25" "X4"  "X13" "X30" "X10"
```

```
(size_suggested <- suggest_size(varsel_pro))
```

```
## [1] 2
```

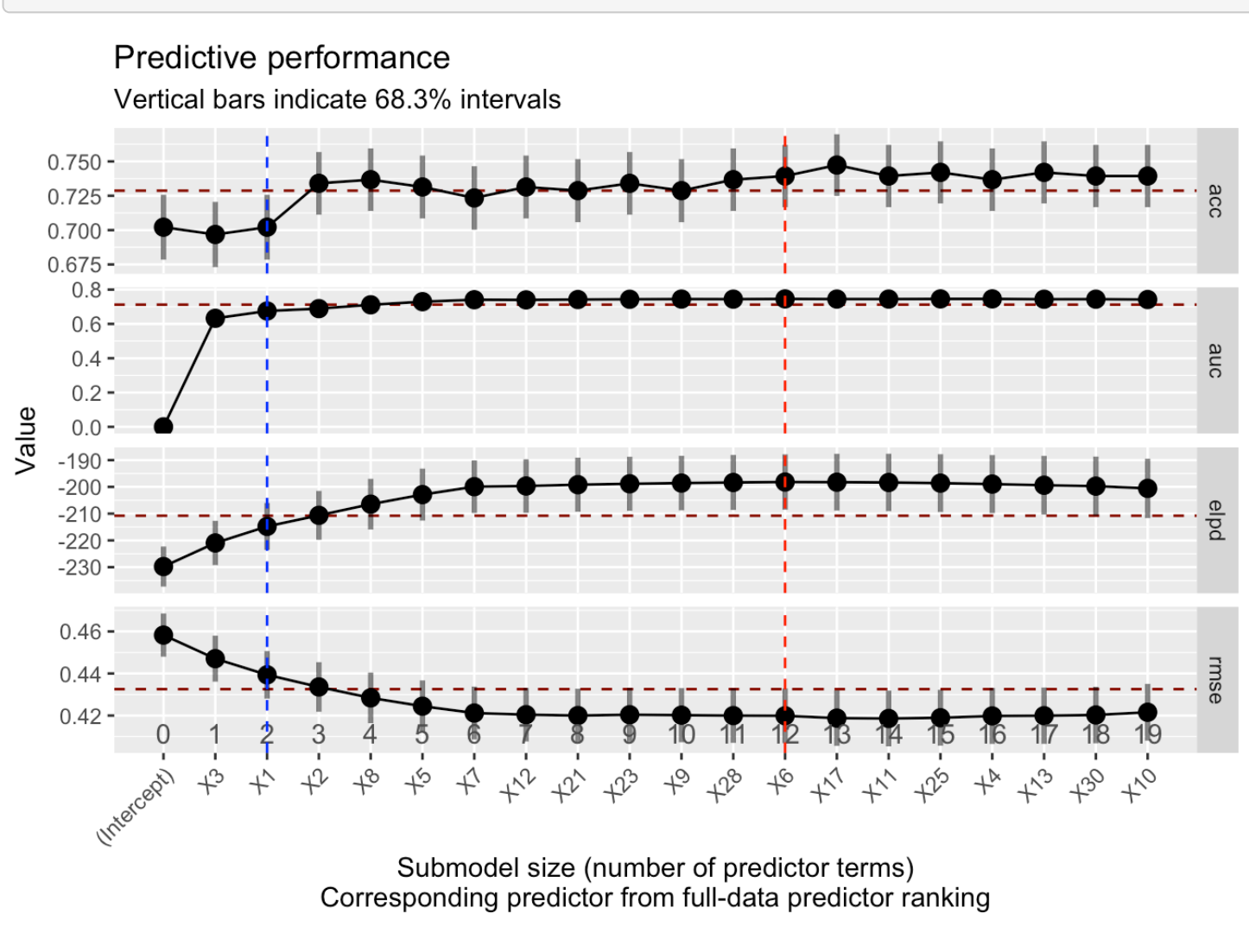
The default suggested size was 2 variables. I compared the performance of submodels with different sizes using different metrics (blue for the default suggested size, red for the size where ELPD was highest) to determine the final set of variables to include in the model.

```
library(pander)
s <- summary(
  varsel_pro,
  stats = c("elpd", "acc", "auc", "rmse"),
  deltas = FALSE
)
performances(s)$submodels %>%
  dplyr::select(size, elpd, elpd.se, elpd.diff, elpd.diff.se, acc, acc.se, auc, auc.se, rmse, rmse.se) %>%
  pander::pander(
    split.cell = 80,
    split.table = Inf,
    justify = "left",
    caption = "Feature selection trajectory statistics for the projection submodels"
  )
```

Feature selection trajectory statistics for the projection submodels

size	elpd	elpd.se	elpd.diff	elpd.diff.se	acc	acc.se	auc	auc.se	rmse	rmse.se
0	-229.8	7.493	-19.01	9.919	0.7021	0.02362	0	0	0.4583	0.01027
1	-221	8.28	-10.19	8.9	0.6968	0.02374	0.6326	0.03173	0.4471	0.01094
2	-214.8	8.749	-4.006	8.179	0.7021	0.02362	0.6753	0.03075	0.4394	0.01131
3	-210.6	9.132	0.1297	7.602	0.734	0.02282	0.6892	0.03085	0.4336	0.01178
4	-206.5	9.467	4.308	7.062	0.7367	0.02274	0.7114	0.02978	0.4284	0.01205
5	-202.9	9.7	7.883	6.629	0.7314	0.02289	0.7296	0.02816	0.4245	0.01226
6	-199.9	9.824	10.84	5.555	0.7234	0.0231	0.741	0.02771	0.4212	0.01258
7	-199.7	10.01	11.09	5.349	0.7314	0.02289	0.7401	0.02762	0.4204	0.01273
8	-199.2	10.1	11.58	5.072	0.7287	0.02296	0.7421	0.02764	0.42	0.01277
9	-198.8	10.1	11.92	4.854	0.734	0.02282	0.7437	0.02785	0.4204	0.01277
10	-198.6	10.18	12.19	4.46	0.7287	0.02296	0.7448	0.02727	0.4202	0.01287
11	-198.3	10.27	12.42	4.209	0.7367	0.02274	0.7444	0.02811	0.42	0.01296
12	-198.2	10.34	12.58	3.996	0.7394	0.02267	0.7458	0.02734	0.4199	0.01303
13	-198.2	10.6	12.52	3.605	0.7473	0.02244	0.7449	0.02811	0.4188	0.01319
14	-198.3	10.73	12.42	3.441	0.7394	0.02267	0.7449	0.02733	0.4186	0.01325
15	-198.6	10.81	12.17	3.246	0.742	0.02259	0.7458	0.02796	0.419	0.01328
16	-199	10.84	11.81	2.906	0.7367	0.02274	0.746	0.02806	0.4198	0.01331
17	-199.4	10.96	11.38	2.601	0.742	0.02259	0.7439	0.02808	0.4199	0.0134
18	-199.7	11.04	11.04	2.216	0.7394	0.02267	0.7443	0.02853	0.4203	0.01345
19	-200.6	11.08	10.21	2.013	0.7394	0.02267	0.7422	0.02752	0.4216	0.0135

```
perf <- performances(varsel_pro)$submodels
size_best <- perf$size[which.max(perf$elpd)]
plot(varsel_pro, stats = c("elpd", "acc", "auc", "rmse")) +
  geom_vline(xintercept = size_suggested, color = "blue", linetype = "dashed") +
  geom_vline(xintercept = size_best, color = "red", linetype = "dashed")
```



I found that there was an elpd “bulge” from size 6. That is, from size 6, the elpd point estimate and its one standard error interval lied completely above the point estimate of the reference model elpd, indicating potentially over-optimistic elpd estimates. We therefore include the search in the CV, yielding more reliable elpd estimates and uncertainties for the different model sizes.

I performed projection predictive variable selection using forward search and 10-fold cross-validation.

```
  varsel_proj <- cv_varsel(ref_fit,
                     method = 'forward',
                     cv_method = "kfold",
                     validate_search = T,
                     K = 10,
                     cores = 8,
                     seed = 11)
```

After projection predictive variable selection, I checked the variable importance, as well the the suggested model size.

```
(vif <- ranking(varsel_proj)$fulldata)
```

```
##  [1] "X3"  "X1"  "X2"  "X8"  "X5"  "X7"  "X12" "X21" "X23" "X9"  "X28" "X6" 
## [13] "X17" "X11" "X25" "X4"  "X13" "X30" "X10"
```

```
(size_suggested <- suggest_size(varsel_proj))
```

```
## [1] 4
```

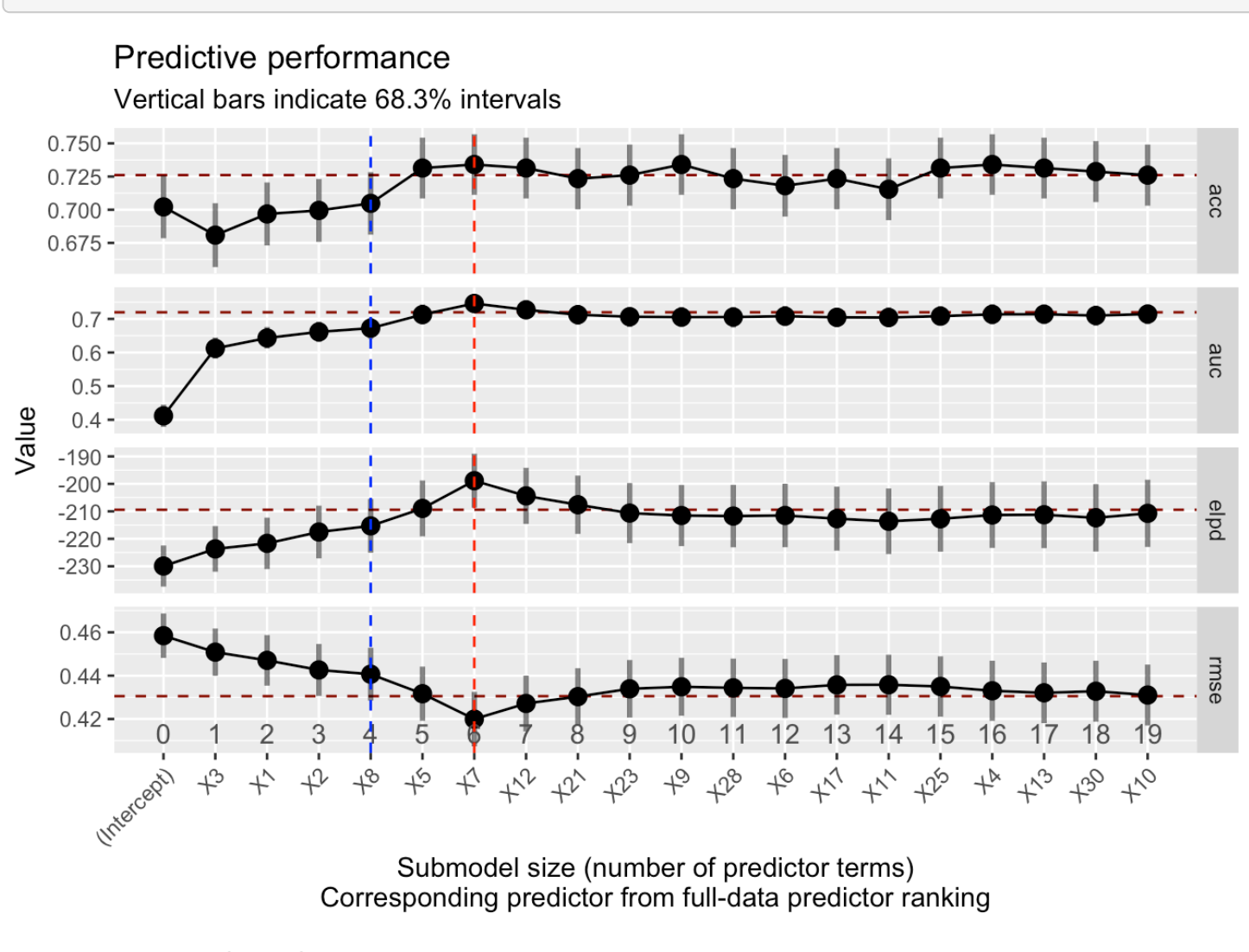
The default suggested size was 4 variables. I compared the performance of submodels with different sizes using different metrics (blue for the default suggested size, red for the size where ELPD was highest) to determine the final set of variables to include in the model.

```
library(pander)
s <- summary(
  varsel_proj,
  stats = c("elpd", "acc", "auc", "rmse"),
  deltas = FALSE
)
performances(s)$submodels %>%
  dplyr::select(size, elpd, elpd.se, elpd.diff, elpd.diff.se, acc, acc.se, auc, auc.se, rmse, rmse.se) %>%
  pander::pander(
    split.cell = 80,
    split.table = Inf,
    justify = "left",
    caption = "Feature selection trajectory statistics for the projection submodels"
  )
```

Feature selection trajectory statistics for the projection submodels

size	elpd	elpd.se	elpd.diff	elpd.diff.se	acc	acc.se	auc	auc.se	rmse	rmse.se
0	-229.9	7.482	-20.49	10.06	0.7021	0.02362	0.4115	0.03248	0.4584	0.01025
1	-223.7	8.313	-14.24	9.149	0.6809	0.02407	0.6123	0.03196	0.4508	0.01092
2	-221.7	9.355	-12.23	8.593	0.6968	0.02374	0.6432	0.0316	0.4471	0.01168
3	-217.5	9.594	-8.097	7.769	0.6995	0.02368	0.6614	0.03024	0.4426	0.01198
4	-215.3	9.788	-5.864	6.884	0.7048	0.02355	0.6725	0.03003	0.4407	0.01219
5	-208.9	10.17	0.4909	6.382	0.7314	0.02289	0.7128	0.02955	0.4317	0.01252
6	-198.8	9.815	10.59	5.788	0.734	0.02282	0.7459	0.02735	0.4199	0.01259
7	-204.4	10.21	5.048	5.482	0.7314	0.02289	0.727	0.0282	0.4272	0.01287
8	-207.6	10.6	1.836	5.158	0.7234	0.0231	0.7125	0.02903	0.4303	0.01312
9	-210.6	10.93	-1.21	4.688	0.7261	0.02303	0.7064	0.02841	0.4339	0.01328
10	-211.5	11.13	-2.095	4.325	0.734	0.02282	0.7053	0.029	0.4349	0.01338
11	-211.7	11.36	-2.294	4.264	0.7234	0.0231	0.7056	0.02961	0.4344	0.01349
12	-211.5	11.55	-2.081	3.711	0.7181	0.02323	0.708	0.02904	0.4341	0.01363
13	-212.7	11.65	-3.237	3.416	0.7234	0.0231	0.7046	0.02889	0.4358	0.01372
14	-213.6	11.94	-4.186	3.468	0.7154	0.0233	0.7041	0.03037	0.4358	0.01388
15	-212.7	11.96	-3.279	3.016	0.7314	0.02289	0.7082	0.02944	0.435	0.01394
16	-211.3	11.98	-1.904	2.824	0.734	0.02282	0.7136	0.02891	0.433	0.0139
17	-211.3	12.16	-1.819	2.496	0.7314	0.02289	0.7139	0.02914	0.4321	0.01401
18	-212.4	12.27	-2.926	2.212	0.7287	0.02296	0.71	0.02948	0.4328	0.01407
19	-210.7	12.23	-1.298	1.991	0.7261	0.02303	0.7144	0.02886	0.431	0.01407

```
perf <- performances(varsel_proj)$submodels
size_best <- perf$size[which.max(perf$elpd)]
plot(varsel_proj, stats = c("elpd", "acc", "auc", "rmse")) +
  geom_vline(xintercept = size_suggested, color = "blue", linetype = "dashed") +
  geom_vline(xintercept = size_best, color = "red", linetype = "dashed")
```



I observed that AUC, RMSE and ELPD reached their best performance at a model size of 6. Accuracy at this model size was also similar to that of the reference model. The suggested model size of 4 was selected using the 1-SE rule, which identifies the smallest submodel whose predictive performance is not statistically distinguishable from that of the best-performing submodel.

Projection

Given the above results, I performed projection of the reference model’s posterior onto submodels with 4 and 6 features.

```
proj_suggest <- project(varsel_proj,
               nterms = size_suggested,
               seed = SEED, ns = 2000)
proj_best <- project(varsel_proj,
               nterms = size_best,
               seed = SEED, ns = 2000)
```

I calculated the predicted probability based on the submodel using the proj_linpred function. I set integrated = TRUE to average over the projected posterior draws and transform = TRUE to obtain predictions on the probability scale.

```
pred_suggest <- proj_linpred(proj_suggest,
                    newdata = data.val[,-1],
                    integrated = TRUE,
                    transform = TRUE
)
pred_best <- proj_linpred(proj_best,
                    newdata = data.val[,-1],
                    integrated = TRUE,
                    transform = TRUE
)
```

Performance

I calculated the performance metrics of these models and compared them with other frequentist models.

```
p_suggest <- pred_suggest$pred[1,]
meas_suggest <- measures(yval, p_suggest)
suggest_stat <- c(size_suggested, meas_suggest)
cat("calibration slope, calibration in the large, auc, rmse values:", meas_suggest)
```

```
## calibration slope, calibration in the large, auc, rmse values: 1.163805 -0.1014574 0.7511986 0.4179602
```

```
p_best <- pred_best$pred[1,]
meas_best <- measures(yval, p_best)
best_stat <- c(size_best, meas_best)
cat("calibration slope, calibration in the large, auc, rmse values:", meas_best)
```

```
## calibration slope, calibration in the large, auc, rmse values: 1.035821 -0.05668834 0.7680126 0.4112255
```

```
# frequentist full model
fullfit <- glm(y ~ ., data = data.dev, family = 'binomial')
eta_full <- as.matrix(cbind(1,xval))%*%coef(fullfit)
p_full <- as.vector(1/(1+exp(-eta_full)))
full_stat <- c(30, measures(yval, p_full))

# BE with p-value threshold of 0.05
p_threshold = 0.05
back_05 <- back_logit(data.dev, "y", "LRT", 0.05)
varsel_back_05 <- back_05$varsel_back
backmodel_05 <- back_05$backmodel
back_eta_05 <- as.matrix(cbind(1,xval[,varsel_back_05 == 1]))%*%coef(backmodel_05)
back_p_05 <- as.vector(1/(1+exp(-back_eta_05)))
back_stat <- c(sum(varsel_back_05), measures(yval, back_p_05))

# UVS with p-value threshold of 0.05
p_threshold <- 0.05
uni_05 <- unilogit(x, y, p_threshold)
varsel_uni_05 <- uni_05$varsel_uni
unimodel_05 <- uni_05$uni
uni_eta_05 <- as.matrix(cbind(1, xval[, varsel_uni_05 == 1])) %*% coef(unimodel_05)
uni_p_05 <- as.vector(1 / (1 + exp(-uni_eta_05)))
uni_stat <- c(sum(varsel_uni_05), measures(yval, uni_p_05))

# LASSO-min
lasso <- glmnet::cv.glmnet(as.matrix(x), y, alpha = 1, family = "binomial", type.measure = "deviance")
lambda_min <- lasso$lambda.min
lassomin_p <- as.vector(predict(lasso, as.matrix(xval), s = lambda_min, type="response"))
varsel_min <- ifelse(as.numeric(coef(lasso, s = lambda_min)[-1]) != 0, 1, 0)
LSmin_stat <- c(sum(varsel_min), measures(yval, lassomin_p))

# LASSO-mod
nfolds <- 10
f <- nfolds / (nfolds - 1) - 1
mod_lasso <- mod_penal_ave_foreach(x = as.matrix(x), y = y, bn = 20, method = "lasso", f = f,
                                   parallel = FALSE, nfolds = nfolds, boot = TRUE)
eta_mod_lasso <- as.matrix(cbind(1,xval))%*% mod_lasso$beta.boot
p_mod_lasso <- as.vector(1/(1+exp(-eta_mod_lasso)))
varsel_mod_lasso <- ifelse(as.numeric(mod_lasso$beta.boot)[-1] != 0, 1, 0)
LSmod_stat <- c(sum(varsel_mod_lasso), measures(yval, p_mod_lasso))

# Bayesian reference model
p_ref <- colMeans(posterior_epred(ref_fit, newdata = xval))
ref_stat <- c(30, measures(yval, p_ref))
```

```
model <- c('Bayesian reference model',
           'Projected submodel (suggest)',
           'Projected submodel (best)',
           'full model',
           'BE-0.05',
           'UVS-0.05',
           'LASSO-min',
           'LASSO-mod')
df <- data.frame(
  Model = model,
  do.call(rbind, list(
  ref_stat,
  suggest_stat,
  best_stat,
  full_stat,
  back_stat,
  uni_stat,
  LSmin_stat,
  LSmod_stat))
)
colnames(df)[-1] <- c("Number of predictors","Calibration slope", "Calibration in the large", "AUC", "RMSE")
df[-1] <- round(df[-1], 3)
pander(
  df,
  justify = c("left", rep("center", ncol(df) - 1)),
  caption = "Model performance comparison"
)
```

Model performance comparison (continued below)

Model	Number of predictors	Calibration slope
Bayesian reference model	30	0.885
Projected submodel (suggest)	4	1.164
Projected submodel (best)	6	1.036
full model	30	0.801
BE-0.05	6	0.965
UVS-0.05	7	0.951
LASSO-min	25	1.258
LASSO-mod	26	1.145

Calibration in the large	AUC	RMSE
-0.128	0.777	0.41
-0.101	0.751	0.418
-0.057	0.768	0.411
-0.116	0.776	0.411
-0.038	0.768	0.411
-0.028	0.764	0.413
-0.053	0.78	0.408
-0.066	0.78	0.408